DAW GUARD

Network Traffic Monitoring & Analysis Platform

Project Report

Course: Computer Networks

Submitted by:

Muhammad Dawood

Student ID: BCSF24M030

Department of Computer Science

2026

1. Abstract

DAW GUARD is a web-based network traffic monitoring and analysis platform developed as part of the Computer Networks course. The application enables users to upload CSV-formatted packet capture files and instantly visualize, filter, and analyze network traffic data through an intuitive dark-themed dashboard. Built with Python (Flask) on the backend and rendered via Jinja2 templates, the system provides real-time statistics on packet protocols, data volumes, and service mappings — all without requiring a database or cloud infrastructure. This report documents the project scope, dataset design, technology stack, methodology, challenges encountered, future enhancements, and references.

2. Introduction

Network traffic analysis is a foundational discipline in computer networking and cybersecurity. Administrators and security analysts routinely inspect packet flows to diagnose performance issues, detect intrusions, and audit policy compliance. Professional-grade tools such as Wireshark operate in real time by hooking into network interfaces; however, they require elevated privileges and can be overwhelming for students learning packet structure and protocol behaviour for the first time.

DAW GUARD bridges this gap by allowing learners to work with pre-captured CSV datasets and explore traffic patterns through a browser-based interface. The application maps well-known destination ports to their service names (e.g., port 443 → HTTPS), computes aggregate statistics, and generates human-readable log entries — giving students hands-on exposure to packet analysis without installing heavyweight toolchains.

3. Scope of the Project

3.1 In-Scope

- Upload and parse CSV files containing network capture data
- Display packet records in a paginated, sortable table
- Filter packets by Protocol (TCP / UDP / ICMP), Source IP, Destination IP, and Service
- Automatically map destination port numbers to standard service names
- Compute and display live statistics: total packets, protocol breakdown, average packet size, total data volume
- Generate terminal-style log records for the most recent packets
- Responsive dark-themed UI with sidebar navigation
- Stateless, single-machine deployment — no external database required

3.2 Out-of-Scope

- Live packet capture from a physical or virtual network interface

- Persistent storage or historical trending across sessions
- User authentication and multi-user support
- Alerting, threat-detection, or IDS/IPS integration
- Export of filtered results to CSV or PDF
- IPv6, ARP, or application-layer deep-packet inspection

4. Dataset Description (CSV Format)

Because DAW GUARD does not capture live traffic, it relies on user-supplied CSV files that represent packet captures. The project ships with a sample dataset (data/sample_traffic.csv) containing 300 synthetic packet records designed to exercise all filtering and statistics features.

4.1 Required Columns

Column	Data Type	Description	Example
Time	String / Timestamp	Packet capture timestamp	2024-01-15 10:23:45.001
Source IP	String (IPv4)	Sender IP address	192.168.1.10
Destination IP	String (IPv4)	Receiver IP address	10.0.0.1
Protocol	String	Transport protocol	TCP
Size	Integer	Packet size in bytes	512
Source Port	Integer	Sender port number	54321
Dest Port	Integer	Destination port number	443

The **Service** column is derived automatically by the backend via the PORT_SERVICE_MAP lookup table — it does not need to be present in the uploaded file.

4.2 Sample Dataset Properties

- 300 packet records covering a typical office LAN scenario
- Protocols: TCP (approx. 55%), UDP (approx. 30%), ICMP (approx. 15%)
- Destination ports sampled from: 21 (FTP), 22 (SSH), 53 (DNS), 80 (HTTP), 443 (HTTPS), 3306 (MySQL)
- Packet sizes uniformly distributed between 64 bytes and 1,500 bytes (standard Ethernet MTU)
- Source IPs drawn from RFC 1918 private address space (192.168.x.x, 10.x.x.x)
- Timestamps spanning a simulated 10-minute capture window

4.3 Port-to-Service Mapping Table

Port	Service	Port	Service
21	FTP	443	HTTPS
22	SSH	445	SMB
23	Telnet	3306	MySQL
25	SMTP	3389	RDP
53	DNS	5432	PostgreSQL
80	HTTP	6379	Redis
110	POP3	8080	HTTP-Alt
143	IMAP	27017	MongoDB
123	NTP	8443	HTTPS-Alt

5. Technology Stack

Layer	Technology	Version / Notes	Role
Language	Python	3.7+	Core backend logic and data processing
Web Framework	Flask	2.x	HTTP routing, request handling, server
Data Processing	Pandas	1.x / 2.x	CSV ingestion, column validation, iteration
Templating	Jinja2	Bundled with Flask	Dynamic HTML rendering (server-side)
Cross-Origin	Flask-CORS	Latest	CORS headers for API routes
Frontend	HTML5 / CSS3	Vanilla	UI layout, dark theme, responsive design
Icons	Font Awesome	6.4.0 (CDN)	Sidebar navigation icons
File Handling	Werkzeug	Bundled with Flask	Secure file upload utilities

No JavaScript frameworks (React, Vue, Angular) or external databases were used. The deliberate choice of a minimal stack keeps the project accessible to students who are primarily studying networking rather than full-stack development.

6. Methodology

6.1 Architecture Overview

DAW GUARD follows a classic Model-View-Controller (MVC) pattern implemented entirely within Flask. The **Model** layer consists of plain Python functions (process_csv_data, calculate_stats, generate_logs, get_unique_services). The **View** layer is the single Jinja2 template (templates/index.html). The **Controller** layer is handled by Flask route functions (@app.route).

The project followed a server-side rendering (SSR) architecture where Flask handles all business logic and Jinja2 templates generate dynamic HTML content

6.2 Data Flow

Step 1: Data Upload

- User uploads a CSV file via the web interface
- Flask receives and validates the file
- Pandas reads and processes the dataset

Step 2: Data Processing

- Required columns are verified
- Each row is converted into a structured packet object
- Destination port is mapped to a service

Step 3: Data Storage

- Data is temporarily stored in memory (no database used)

Step 4: Data Visualization

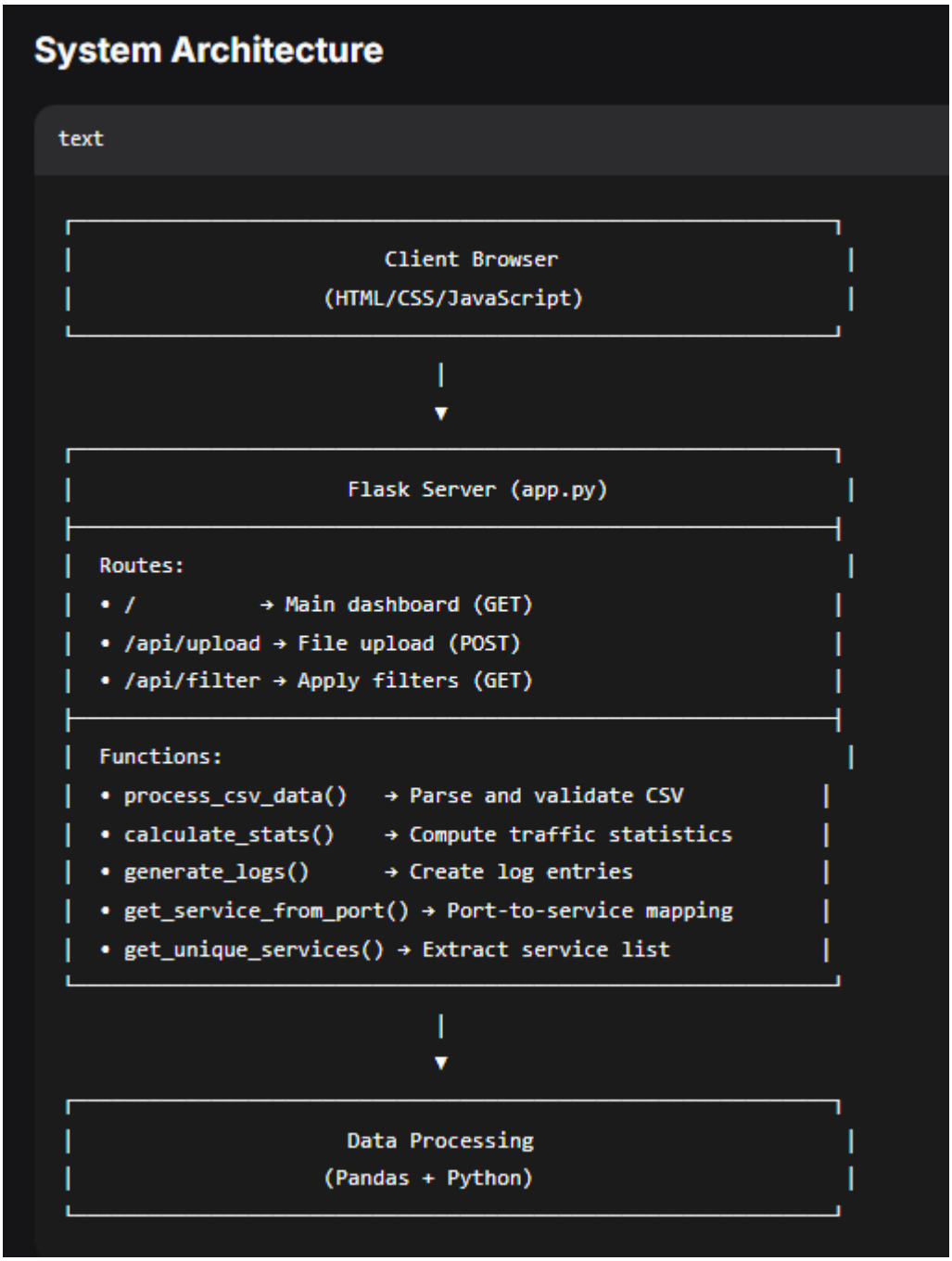
- Packets are displayed in a table
- Statistics are calculated:
  - Total packets
  - Protocol distribution
  - Average packet size

Step 5: Filtering

- Users can filter packets by:
  - Protocol
  - Source IP
  - Destination IP
  - Service

Step 6: Logging

- Logs are generated in a terminal-style format for readability



6.3 Key Functions

Function	Location	Purpose
process_csv_data(df)	app.py	Validates columns; converts DataFrame rows to packet dicts; maps ports to services

Function	Location	Purpose
calculate_stats(packets)	app.py	Computes total, TCP/UDP/ICMP counts, avg size, total size
generate_logs(packets, limit)	app.py	Produces formatted log strings for the last N packets
get_unique_services(packets)	app.py	Returns sorted set of service names for filter dropdown
get_service_from_port(port)	app.py	Looks up PORT_SERVICE_MAP; returns 'Port N' fallback
allowed_file(filename)	app.py	Whitelists .csv extension for upload security

6.4 Frontend Design Decisions

The UI adopts a cybersecurity aesthetic — dark navy backgrounds (#1A1A2E) contrasted with neon orange (#E85D04) accents — inspired by professional SIEM dashboards. The layout uses CSS Grid and Flexbox to create a three-zone composition: a fixed left sidebar for navigation, a wide central pane containing filters, the traffic table, and logs, and a narrow right sidebar for the statistics panel.

Protocol badges (TCP / UDP / ICMP) are colour-coded via CSS classes to allow instant visual scanning of packet types in the table. Font Awesome icons are loaded from CDN to avoid build tooling. The initial UI layout concept was guided by mockups generated with ChatGPT, which helped establish the colour palette, card structure, and sidebar navigation pattern before any code was written.

7. Challenges Faced

7.1 Time Shortage

The project was developed within a compressed academic timeline alongside other coursework. This necessitated strict scope control — features such as live packet capture and persistent storage were deliberately excluded so that a polished, working MVP could be delivered on time. Prioritisation frameworks (must-have vs. nice-to-have) were applied to every design decision. Daily micro-milestones (e.g., 'CSV upload working today', 'filter route tomorrow') kept progress on track despite the tight schedule.

7.2 No Frontend–Backend Integration Experience

Before this project, experience with full-stack web development was limited. Connecting Python server logic to a live HTML page required learning Flask's request/response cycle, Jinja2 template context injection, and HTML form methods (GET vs. POST). Debugging issues such as the filter form losing its state after navigation, or the service dropdown not pre-selecting the current filter value, consumed significant development time. Understanding the difference between a POST redirect and a direct template render was a key learning milestone.

7.3 UI Design Without a Design Background

Designing a visually appealing cybersecurity-themed dashboard from scratch, using only vanilla HTML and CSS (no component library), was challenging. Achieving the dark-theme card layout with proper overflow handling, responsive columns, and consistent spacing required iterative CSS work. AI-generated UI mockups (via ChatGPT) were used as visual references to guide the colour palette (#1A1A2E navy and #E85D04 neon orange) and layout structure. Without this reference, achieving a professional SIEM-like aesthetic would have taken significantly longer.

7.4 CSV Column Validation and Error Handling

Handling arbitrary CSV uploads robustly required defensive programming: checking for missing columns, catching type-conversion errors (e.g., non-integer port values), and surfacing meaningful error messages to the user without crashing the Flask server. The process_csv_data() function was refactored several times to harden this validation logic. Edge cases such as trailing spaces in column names, mixed-case protocol strings, and BOM-encoded CSV files were discovered only through real test uploads.

7.5 State Management Without a Database

Storing the uploaded packet list in a Python global variable (packets_data) is simple but fragile: the data is lost on server restart, and concurrent users would overwrite each other's data. For an academic single-user tool this trade-off was acceptable, but it highlighted the importance of session management and persistent storage in production web applications.

8. Future Work

8.1 Live Packet Capture

Integrating the Scapy or PyShark library would allow DAW GUARD to capture packets directly from a network interface in real time. Combined with WebSocket push (Flask-SocketIO), the table and statistics panel could update live without page reloads, transforming the tool into a genuine monitoring appliance.

8.2 Threat Detection & Anomaly Alerts

Rule-based or machine-learning anomaly detection could flag port scans, SYN flood patterns, unusually large packets, or traffic to suspicious IP ranges. Visual alerts (badge indicators, toast notifications) and email/Slack notifications could be added to make DAW GUARD a lightweight Intrusion Detection System (IDS).

8.3 Persistent Storage & Historical Analytics

Persisting captures to an SQLite or PostgreSQL database would enable cross-session querying, trend analysis over time, and comparison between multiple capture files. Time-series charts (using Chart.js or Plotly) could visualise traffic volume, protocol distribution, and top-talker IP addresses across configurable time windows.

8.4 User Authentication & Multi-Tenancy

Adding Flask-Login with hashed passwords would allow multiple analysts to maintain separate capture sessions. Role-based access control (RBAC) could differentiate read-only viewers from administrators who can delete captures or manage users.

8.6 Export & Reporting

A one-click export button that produces a filtered CSV or PDF report would allow analysts to share findings. Automatically generated executive-summary PDF reports — listing top source IPs, busiest services, and any anomaly flags — would add significant practical value.

9. System Requirements & Installation

9.1 Hardware & Software Prerequisites

Component	Requirement
Operating System	Windows 10/11, macOS 12+, or Ubuntu 20.04+
Python	3.7 or later
pip	Latest (comes with Python 3.7+)
Browser	Chrome 90+, Firefox 88+, Edge 90+ (any modern browser)
RAM	Minimum 512 MB free (for datasets up to 50 MB)
Disk	Minimum 100 MB for dependencies

9.2 Installation Commands

```
git clone https://github.com/your-username/network-monitor.git
cd network-monitor
pip install -r requirements.txt
python app.py
# Open browser at: http://127.0.0.1:5000
```

9.3 Python Dependencies (requirements.txt)

- flask
- flask-cors
- pandas
- werkzeug

10. Conclusion

DAW GUARD successfully demonstrates the practical application of Computer Networks concepts — specifically TCP/IP protocol stack, port-to-service mappings, and packet metadata — within a deployable web application. The project moved from a blank Python file to a fully functional, visually polished monitoring dashboard, providing hands-on learning in Flask web development, Pandas data processing, Jinja2 templating, and CSS-driven UI design.

Despite constraints of time and limited prior frontend-backend integration experience, all core features were delivered: CSV upload and validation, protocol/IP/service filtering, real-time statistics, and log generation. AI tools — ChatGPT for UI design inspiration and Claude Haiku 4.5 for coding assistance — played a meaningful supporting role in accelerating development quality. The identified future enhancements — live capture, anomaly detection, persistent storage, and PCAP support — provide a clear roadmap for evolving DAW GUARD into a production-grade network analysis tool.

This project reinforced that network security tooling need not be confined to expensive commercial products; with a focused scope and the right open-source libraries, a developer can build genuinely useful monitoring capabilities from first principles.

11. References

[1] OpenAI ChatGPT — Used for generating UI layout concepts, colour palette exploration, and CSS design inspiration for the dark-themed cybersecurity dashboard. Available: https://chat.openai.com

[2] Anthropic Claude Haiku 4.5 — Employed as an AI coding assistant during development for debugging Flask routing, Jinja2 template syntax, and CSS layout issues. Available: https://claude.ai

[3] Flask Documentation (Pallets Projects). "Flask — A Lightweight WSGI Web Application Framework." Available: https://flask.palletsprojects.com

[4] Pandas Development Team. "pandas — Powerful Python Data Analysis Toolkit." Available: https://pandas.pydata.org/docs

[5] Jinja2 Documentation (Pallets Projects). "Jinja — Template Engine for Python." Available: https://jinja.palletsprojects.com

[6] Font Awesome. "Font Awesome Icon Library v6.4.0." Available: https://fontawesome.com

[7] Forouzan, B. A. (2012). Data Communications and Networking, 5th Edition. McGraw-Hill. (Used as theoretical reference for TCP/IP protocol stack and port assignments.)

[8] IANA. "Service Name and Transport Protocol Port Number Registry." Available: https://www.iana.org/assignments/service-names-port-numbers

[9] Mozilla Developer Network (MDN). "HTTP — HyperText Transfer Protocol." Available: https://developer.mozilla.org/en-US/docs/Web/HTTP

[10] Werkzeug Documentation. "Werkzeug WSGI Utility Library." Available: https://werkzeug.palletsprojects.com